

DALOC

A Cross-Chain Prime Brokerage Protocol
for Collateralized Credit

Bartosz Nowak	Marcello Bardus	Jamil Bouhdada
bartosz@herodotus.dev	marcello@herodotus.dev	jamil@herodotus.dev

June 2026

Important Notice. This document describes the technical architecture of a protocol under development. It does not constitute an offer, solicitation, or recommendation to purchase any token, security, financial instrument, or investment product. The system described may never be deployed, or may differ materially from this description. Nothing herein constitutes financial, investment, or legal advice. References to third-party protocols are for technical context only and do not imply endorsement or partnership. Readers should consult independent professional advisors before taking any action based on this document.

Abstract

DALOC is a cross-chain credit coordination and settlement protocol for prime brokerage and structured lending. A single collateral deposit provides leverage and risk management across multiple chains and venues without bridging the underlying asset. Collateral stays on its home chain; yield-bearing tokens continue accruing native yield. Coordination relies on zero-knowledge storage proofs rather than trusted relayers, while token movement uses existing bridge infrastructure.

Three products share one protocol, built around independently operated markets that anyone can deploy. Borrow originates fixed-rate, fixed-term loans against collateral on any supported chain. Prime Trade opens a credit line from collateral to a perpetuals venue, where borrowed funds are restricted to trading by market-defined margin agents. Earn gives liquidity providers yield from a portfolio of fixed-rate loans: capital is deployed by a vault operator whose strategy is bounded by a proof-attested mandate enforced through fee-gating.

A common order book structure matches borrower intents against solver quotes within each market; criteria-based filtering lets regulated and open participants share one liquidity surface without permissioned forks. Capital is sourced through liquidity forwarding: solvers borrow from established lending venues against pledged collateral, reserving only hedging capital to convert variable funding to fixed terms; the protocol can originate loans before accumulating LP deposits. Markets are defined by permissionless zero-knowledge programs, and risk terms are per-loan, dynamically simulated, and zero-knowledge provable. Loan positions are tokenized as transferable debt instruments with provably accruing interest.

Contents

1	Introduction	5
2	Products	6
2.1	Borrow	6
2.2	Prime Trade	6
2.3	Earn	6
3	Protocol Architecture	7
3.1	Order Book	7
3.2	Criteria-Based Matching	7
3.3	Markets	7
3.4	Hook Composition	8
3.5	Programmable Credit	9
3.6	Solvers	10
4	Capital Management	10
4.1	Vault-Funded Path	10
4.2	Liquidity-Forwarding Path	10
4.3	Capital Efficiency	11
5	Loan Lifecycle	12
5.1	Origination	12
5.2	Refinancing	13
5.3	Close and Liquidation	13
6	The Harvest Vault	15
6.1	Operator and Mandate	16
6.2	Compliance Proof	16
6.3	Fee-Gating	16
6.4	NAV and Shares	17
6.5	Epoch Withdrawal	17

7	Risk Quantification	17
7.1	Risk Objective and Loss Definition	18
7.2	Dynamic Underwriting	19
7.3	Scenario Generation	20
7.4	Provable Risk Engine	20
7.5	Funding and Execution Reserves	21
7.6	Pricing	21
7.7	Portfolio Aggregation and Concentration Risk	22
7.8	Risk Houses and Liquidation Assurance	22
7.9	Operational Risk and Model Governance	23
8	Cross-Chain Exposure	23
8.1	Authorization Flow	24
8.2	Deterministic Addressing	24
8.3	Hyperliquid Margin Agent	24
8.4	Effective Leverage	24
9	Settlement and Verification	25
9.1	Epoch Mechanics	25
9.2	Settlement Programs	26
9.3	Market and Loan Contracts	26
9.4	Oracle Freedom and Permissionless Markets	27
9.5	System Invariants	27
10	Position Tokens	27
10.1	Debt Tokens	27
11	Governance and Trust Assumptions	28
11.1	Protocol Administration	29
11.2	Liveness Dependencies	29
12	Discussion	30
12.1	Risks and Limitations	30
12.2	Future Capabilities	31

12.3 Conclusion	32
---------------------------	----

1 Introduction

A holder of yield-bearing collateral on one chain who wants leveraged exposure on another must unstake, bridge, and re-deposit, giving up yield and taking on counterparty risk at each step. On-chain lending exceeds \$28B [12], yet the dominant pool-based designs serve only variable-rate, open-term use cases on a single chain.

Existing protocols address important fragments of this problem. Aave provides large-scale pooled lending with governance-managed risk parameters, isolation modes, e-mode, and a hub-and-spoke architecture in Aave V4 [1]. Euler V2 provides a modular lending framework through isolated vaults and the Ethereum Vault Connector, letting builders compose customized credit markets [14]. Morpho Blue separates lending infrastructure from risk curation through isolated markets, while Morpho’s fixed-maturity work moves closer to intent-based, fixed-term credit backed by tokenized debt [15, 24]. Spark focuses on conservative stablecoin lending and protocol-directed capital allocation through SparkLend and related liquidity infrastructure [30]. Vault-based systems further improve allocation by curating exposure across markets [34].

These systems improve capital efficiency, market customization, and risk curation. None simultaneously coordinates fixed-rate credit across chains, keeps collateral on its home chain while proving its state without trusted relayers, sources capital from external lending venues, converts variable funding into fixed terms, routes borrowed capital into constrained execution environments, and accommodates institutional criteria without fragmenting liquidity. Public materials and roadmaps for the major lending protocols show overlap with individual components, especially modular markets, curated vaults, fixed-term loans, and cross-chain architecture, but not this full combination inside one settlement and verification layer [1, 14, 15, 24, 30].

DALOC unifies these capabilities in a single credit coordination and settlement layer. Fixed-rate loans, leveraged perpetual trading, and passive yield share a common matching and settlement infrastructure; borrowers, traders, and liquidity providers interact through the same intent structure without bridging collateral or fragmenting liquidity. Capital is sourced from established lending venues via liquidity forwarding; criteria-based filtering accommodates regulated participants alongside open ones. Programmable zero-knowledge markets, a provable risk engine, and solver-driven execution complete the architecture; independently operated markets isolate liveness and trust assumptions. The architecture carries inherent risks (smart-contract vulnerabilities, oracle dependencies, bridge liveness, and proof-system availability) discussed in Sections 11 and 12.

2 Products

All three products share a single protocol layer (the Harvest Vault, per-market order book, settlement pipeline, and risk engine), extensible to new products without modifying core contracts.

2.1 Borrow

A holder of any supported token borrows against it at a fixed rate for a fixed term. Collateral stays on its home chain; yield-bearing collateral (e.g. wstETH) continues accruing native yield throughout the loan. Loan-to-value ratio, rate, and term are determined per loan through solver competition (Section 3). Origination, interest accrual, and liquidation are each governed by zero-knowledge programs, ensuring that no single trusted party controls the loan’s lifecycle.

2.2 Prime Trade

A borrower draws a credit line in the market’s supported principal asset and routes it to a per-loan margin agent on HyperEVM, gaining leveraged perpetual exposure without selling the underlying collateral (Section 8). In the initial markets, USDC is the canonical principal asset because of its deep DeFi liquidity. USDC is therefore the reference implementation, not a permanent protocol constraint: other stablecoins may be supported by separate markets if the relevant liquidity, risk, and compliance requirements are satisfied.

The margin agent restricts capital to approved operations (trading only, no withdrawals), so the lender’s risk depends on programmatic constraints, not borrower discretion. Venue leverage multiplies the borrowing LTV, controlling large notional with modest collateral (Section 8).

2.3 Earn

Liquidity providers deposit USDC into the Harvest Vault, where an operator deploys capital into fixed-rate loans originated through the protocol’s order book, either funding loan principal directly or providing hedging buffers when loans are sourced through external venues (Section 4). The operator’s strategy is bounded by a publicly auditable mandate; compliance is attested per-epoch by zero-knowledge proof and enforced through fee-gating (Section 6); the operator’s fee is released only upon valid compliance proof. On vault-funded loans, yield derives from the full fixed-rate coupon; on liquidity-forwarding loans, from the hedging-buffer surplus. Withdrawal is epoch-based with a settlement delay.

3 Protocol Architecture

3.1 Order Book

All loan activity flows through a single order book maintained off-chain by each market's operator (the party that advances epochs, performs matching, and produces zero-knowledge proofs for that market). Borrower requests and solver quotes use the same intent structure: collateral, principal, rate, term, and execution path (the full quote specification is given in Section 7). The operator selects the best rate for the borrower among criteria-compatible quotes; the settlement proof verifies that no better compatible match was skipped (without revealing other quotes' terms). Quote terms are visible only to the operator, the solver, and the borrower.

Participation requires no upfront capital lock. A borrower or solver signs an off-chain intent and grants a token allowance to the hub, authorizing a future transfer but moving nothing. Capital is transferred only when an epoch's settlement proof executes the match on-chain; until then, funds remain in the participant's wallet.

3.2 Criteria-Based Matching

Beyond price compatibility, a match must satisfy *criteria*: typed attributes attached to an intent (counterparty allowlists, jurisdictional restrictions, KYC-tier requirements, or sanctions-screen attestations). The operator treats a pair as compatible only if each side's criteria are satisfied by the other's attributes. Unrestricted participants cannot match against restricted entries whose criteria they do not satisfy; restricted participants match only against compatible counterparties. There is no separate permissioned market; entries in one order book vary only in how restrictive they are.

This avoids the fragmentation of permissioned forks: one liquidity surface, one risk engine, one settlement pipeline, one audit perimeter. Adding a new jurisdiction means adding a new attestation issuer, not deploying new contracts.

Criteria can be committed-only (stored as a hash, evaluated inside the proof, revealing only the boolean outcome). Counterparty attributes are typically committed-only, preserving privacy while still enforcing compatibility. The default is open: an unrestricted intent imposes no criteria on its counterparty. Criteria proofs are a protocol-level matching primitive; they do not constitute or replace compliance with applicable legal or regulatory obligations, which remain the responsibility of each participant.

3.3 Markets

Criteria restrict *who* can match; markets define *what* the matched position looks like. DALOC is organized around isolated lending markets. Each market is

uniquely identified by a deterministic hash of its parameters: the collateral chain and asset, principal chain and asset, and four zero-knowledge program hashes that govern loan behavior, including accrual, liquidation, collateral policy, and oracle-dependent state transitions. All parameters are fixed at creation; a different rate model, liquidation rule, collateral policy, or oracle policy constitutes a different market.

Oracle selection follows the same immutability rule. In protocols such as Morpho Blue, the oracle is an explicit market parameter: each market points to a specific oracle contract, fixed at creation [15]. In DALOC, oracle logic is embedded in the market's zero-knowledge programs instead. A market program specifies which on-chain sources may be read, how prices or rates are transformed, and which freshness constraints must hold before a state transition is accepted. The protocol verifies the program output, not a protocol-wide oracle choice. Changing the oracle source, aggregation rule, freshness bound, or liquidation-price logic changes the program hash and therefore defines a new market.

Markets are created permissionlessly through the *hub* contract (DalocHub), deployed at identical addresses on every supported chain (Section 8). Each chain's hub instance is the sole token rail on that chain: all user-facing ERC-20 transfers route through it, creating a single per-chain authorization point for capital flows, intent cancellations, and global pause.

Different markets can encode different collateral strategies: a *classical* market locks collateral as a liquidation hedge, while a more aggressive market can permit managed rehypothecation. The choice is a market-definition decision, not a protocol-level toggle.

Each market deploys issuer contracts on both the collateral and principal chains; each funded loan creates per-loan contracts at deterministic CREATE2 addresses. Per-loan contracts are isolated (one per loan, not shared across borrowers), so one position's risk cannot cascade to another.

Isolation extends beyond individual positions to entire markets. Each market operates as an independent state machine: it advances its own epochs, maintains its own state root, and shares no global state with other markets. A market can access only its own balances; one market's bug or misconfiguration cannot reach into another's funds. Multiple markets for the same collateral/principal pair may coexist under different risk parameters, oracle policies, rule sets, or execution operators; participants evaluate each market's operator, parameters, and track record before choosing where to transact.

3.4 Hook Composition

Lending operations are expressed as atomic hook chains attached to settlement events. Each transfer in the settlement tree carries optional *pre-hooks* and *post-hooks*: pre-hooks execute before the transfer (e.g. deploy a CDP), the transfer

moves collateral or principal, and post-hooks execute after (e.g. supply to an external venue, borrow, bridge). All steps on a single chain must succeed or the entire on-chain operation reverts; cross-chain steps (bridge hooks) are committed atomically but execute asynchronously on the destination chain, stalling rather than reverting if the bridge is delayed (Section 8).

Solvers assemble hook chains and commit them into the settlement tree before on-chain execution. The settlement proof verifies that every hook is authorized by the market's ZK programs and that the resulting state satisfies the market's invariants. Borrowers sign high-level intents; solvers compete to attach the cheapest valid execution plan. Intent cancellation is always submittable (even when the protocol is paused) and takes effect once the next epoch settles.

3.5 Programmable Credit

Permissionless markets, hook composition, and constrained execution together form a programmable credit layer. A market creator writes ZK programs governing accrual, liquidation, and collateral policy; solvers compete to assemble the cheapest valid execution plan from hooks; borrowers express only high-level intents.

Constrained execution environments extend this programmability to the destination chain. A margin agent on HyperEVM restricts borrowed capital to approved operations (perpetual trading only, no withdrawals), but the constraint boundary itself is programmable: HyperEVM contracts interact with HyperCore's perpetual engine via precompiles, enabling on-chain strategies within the lender-defined safety envelope:

- **Delta-neutral yield.** A borrower pledges stETH, borrows USDC, and the margin agent automatically opens a short ETH perpetual matching the collateral's notional. The position is market-neutral; net yield equals the staking rate minus funding cost minus the DALOC fixed borrowing rate. The constraint permits only a single, fully hedged position with no directional bets.
- **Automated stop-loss.** The margin agent monitors unrealized PnL via precompile reads and auto-closes the position if losses exceed a configurable threshold, bounding lender exposure during inactive periods.
- **Portfolio rebalancing.** The margin agent maintains multiple perpetual positions at target allocations, rebalancing when drift exceeds a bound and keeping margin utilization within the market's declared limits.

Because the lender's risk depends on what capital *can* do, not on borrower intent, tighter constraints enable more favorable terms and may, in future, support under-collateralized credit lines.

3.6 Solvers

The Harvest Vault operator is the canonical solver, deploying vault capital into loans directly when capacity permits or routing through external venues when vault capacity is insufficient (Section 4). The operator competes on the order book under the same rules as every other solver; the mandate (Section 6) bounds its strategy, not its participation.

Independent solvers coexist on the same book. A *risk-taking* solver funds from its own balance sheet; a *passive* solver performs *liquidity forwarding* by assembling a hook chain that deposits pledged collateral on established venues (Aave [1], Morpho [15], Spark [30]) and borrows against it. Both modes produce structurally identical entries; the best terms win. Routing and capital sourcing are solver-level concerns, making the protocol extensible to new venues without core contract changes. DALOC provides the settlement rails; existing venues provide the liquidity.

4 Capital Management

The Harvest Vault is the canonical capital source. When idle vault capital exceeds the requested principal, the operator funds the loan directly, transferring USDC to the borrower and receiving debt tokens as the creditor of record. When demand exceeds vault capacity, solvers extend reach through external lending venues, reserving vault capital only for the hedging instruments that convert variable-rate external funding to fixed terms. The operator’s mandate (Section 6) governs the allocation between paths.

4.1 Vault-Funded Path

The vault transfers principal directly to the borrower (or to the margin agent for Prime Trade). The CDP holds collateral as a liquidation hedge; no external venue is involved. Capital is committed at 1:1 with no variable-rate mismatch or hedging buffers, and the vault captures the full fixed-rate coupon. This path is preferred whenever vault capacity permits.

4.2 Liquidity-Forwarding Path

When vault idle capital is insufficient (because existing deployments have consumed it, or because the loan size exceeds available balance), solvers source principal from established lending venues rather than from the vault balance sheet directly. The CDP deposits pledged collateral, or an approved intermediate asset after a collateral swap, into an admissible external venue, borrows the market’s supported principal asset against it, and routes the borrowed asset to the borrower or margin agent. For the initial USDC-based markets, admissible venues may include Aave [1], Morpho [15], and Spark [30]. The vault commits only a fraction of principal as hedging reserves (rate buffer plus col-

lateral conversion reserve), achieving high capital leverage. Pledged collateral simultaneously secures the DALOC loan and the external venue position.

External venues are not selected arbitrarily by solvers at execution time. Each market defines its admissible venue set through its zero-knowledge programs and authorized hook adapters. A venue is admissible only if its collateral support, principal asset, oracle behavior, liquidation mechanics, rate model, and on-chain state can be verified by the market’s programs and incorporated into the risk engine. Changing the venue set, adapter, or verification logic therefore defines a different market.

When Harvest Vault capital is used for hedging reserves, the vault mandate may impose stricter constraints than the market itself: allowed venues, maximum exposure per venue, maximum utilization or borrow-rate assumptions, minimum liquidity depth, liquidation-latency bounds, and adapter-level risk limits. The operator’s per-epoch compliance proof attests that liquidity-forwarding activity stayed within this mandate; a missing or invalid proof forfeits the operator’s fee for that epoch (Section 6).

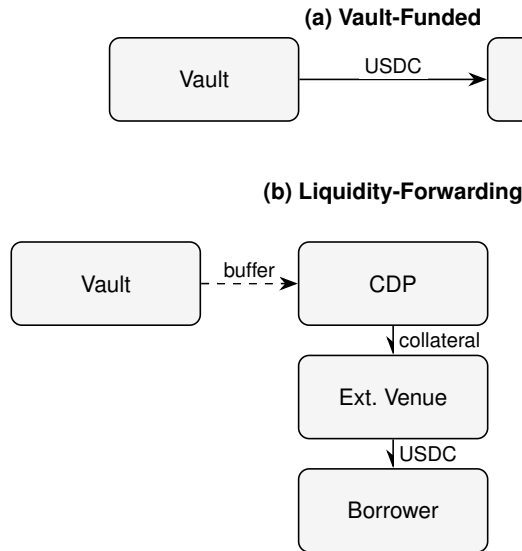


Figure 1: Capital paths. **(a)** Vault-funded: the vault transfers principal directly and captures the full fixed-rate coupon. **(b)** Liquidity-forwarding: the CDP borrows from the external venue; the vault commits only hedging capital (dashed), achieving $\sim 20:1$ leverage.

4.3 Capital Efficiency

Both buffers on the liquidity-forwarding path are sized at quote time by the provable risk engine (Section 7): the rate buffer from tail simulations of variable-rate paths, the collateral conversion reserve from simulated round-trip execu-

tion costs. For a liquidity-forwarding loan, the vault’s capital leverage is

$$\kappa = \frac{P}{B_{\text{rate}} + B_{\text{conv}}}, \quad (4.1)$$

where P is the loan principal and $B_{\text{rate}} + B_{\text{conv}}$ is the total hedging capital committed. On the vault-funded path the vault commits the full principal ($\kappa = 1$ by definition); on the liquidity-forwarding path, a typical rate buffer of $\sim 5\%$ achieves $\kappa \approx 20$. The blend between paths depends on vault capacity and market demand.

At close, buffer surplus remains in the vault as LP yield; buffer shortfall reduces vault NAV.

5 Loan Lifecycle

5.1 Origination

A borrower posts a request, solvers respond with quotes, and the accepted quote is committed into an epoch’s settlement tree and executed atomically. Which capital path applies (Section 4) determines the pipeline complexity. Four origination pipelines exist, each assembled as an atomic hook chain:

Vault-funded. The vault transfers USDC principal directly to the borrower (or to the margin agent for Prime Trade positions); the CDP holds collateral as a liquidation hedge without supplying it to an external venue, and no hedging buffers are reserved.

Liquidity-forwarding. The hook chain deposits collateral into the collateral manager, supplies it to the external venue, borrows USDC against it, and transfers the USDC to the borrower. A rate buffer is escrowed to hedge the mismatch between the borrower’s fixed rate and the venue’s variable rate.

Liquidity-forwarding with collateral swap. For collateral not directly supported by the selected external venue (e.g. LDO or UNI), the market may authorize a swap hook that converts the pledged collateral into a venue-listed intermediate asset (e.g. LDO $\rightarrow$ WETH) through an approved AMM, RFQ venue, or aggregator before the hook chain proceeds as above. This path is available only for collateral assets, intermediate assets, swap adapters, and execution routes admitted by the market’s zero-knowledge programs and, where applicable, by the designated Risk House policy and Harvest Vault mandate.

The swap hook is executed with explicit price-protection constraints committed in the settlement leaf (minimum output, deadline, maximum slippage). If these constraints are not satisfied at execution time, the hook reverts and the origination does not settle. Both a rate buffer and a collateral conversion reserve are escrowed; the conversion reserve is sized by the risk engine to cover stressed round-trip execution costs. In canonical DALOC markets, assets or routes with

insufficient oracle quality, shallow executable liquidity, or unavailable liquidation paths are rejected rather than routed through the swap path.

Hyperliquid margin. For Prime Trade positions, the hook chain follows either pipeline above but diverts at the final step: instead of transferring USDC to the borrower, a bridge hook routes it to a per-loan margin agent on HyperEVM, which deposits it as margin (Section 8). Collateral never leaves Ethereum.

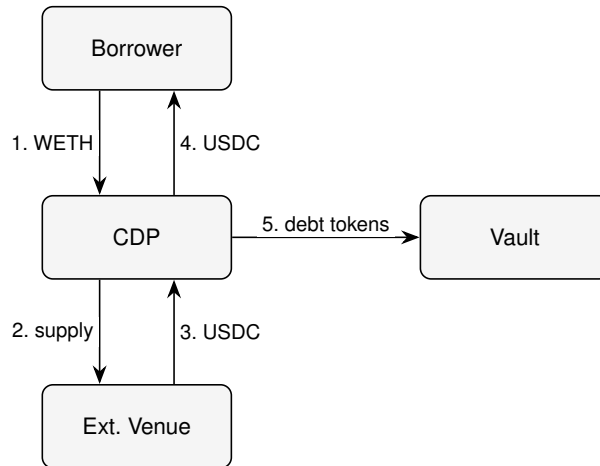


Figure 2: Origination, liquidity-forwarding pipeline. Steps 1–4 execute as a single atomic hook chain. Debt tokens (step 5) are minted to the vault; the rate buffer is escrowed. Vault-funded origination omits the external venue and transfers principal directly.

5.2 Refinancing

Not every loan runs to its original maturity. A borrower refinances by posting a single refinancing intent; the matching engine treats it identically to a new origination request. On match, the existing loan closes and a new one originates atomically within the same epoch. When the Harvest Vault is creditor on both sides of a vault-funded loan, refinancing is a pure accounting event: no capital moves, only terms change. When the creditor differs or the loan involves an external venue, repayment and new funding settle within the same settlement tree. For Prime Trade positions, refinancing applies to the credit terms on Ethereum; margin capital on HyperEVM remains in place unless the position is fully closed and re-originated.

5.3 Close and Liquidation

On voluntary close, the borrower deposits the principal asset into the loan’s debt contract. After interest accrual is verified (Section 10), the creditor (vault or independent solver) redeems its position and a close hook chain repays the variable debt on the external venue, withdraws the collateral, and returns it to the borrower (with a reverse swap for collateral-swap loans). Hedging surplus

stays with the capital provider as yield. Vault-funded loans skip the external unwind: the borrower repays, and collateral is released directly. For Prime Trade positions, the close hook chain first recalls capital from the destination venue via the bridge before repaying the source-chain debt.

The external unwind is subject to venue-level liquidity and liveness constraints. If the venue cannot provide the required borrow liquidity at origination, or if utilization, borrow rate, pause status, or available liquidity violates the bounds committed in the quote, the origination hook reverts and the loan does not settle. For active liquidity-forwarding loans, a stressed venue can increase variable funding costs, delay collateral withdrawal, or force execution through a slower liquidation route. These risks are modeled by the risk engine through rate buffers, collateral-conversion reserves, venue exposure limits, and liquidation-latency assumptions.

If a close or liquidation cannot fully unwind the external venue position in the expected epoch because the venue has insufficient available liquidity, withdrawals are paused, or the adapter reports a failed unwind, the loan enters a pending-unwind state. Debt repayment and recovered assets are ring-fenced, borrower withdrawals are delayed until the external position is unwound or auctioned, and any excess funding or execution cost is charged first against loan-specific reserves. Residual shortfall follows the loss waterfall (Section 7): Risk House assurance commitment, Risk House first-loss capital, any protocol catastrophe layer, and finally vault impairment according to the market policy. DALOC cannot remove external venue liquidity risk; it makes the risk explicit, priced, reserved, and isolated per loan.

Liquidation triggers when the health factor drops below 1:

$$\text{HF} = \frac{C_{\text{val}} \theta}{D} < 1, \quad (5.1)$$

where C_{val} is the collateral's current market value, θ the loan's liquidation threshold (set at origination, strictly above the origination LTV), and D the outstanding debt including accrued interest. Because D grows with interest accrual, the risk engine constrains (θ, λ, r, T) so that the health factor remains above 1 at maturity under constant collateral value.

Liquidation spans two epochs, separating detection from seizure.

Detection (epoch n). Any party generates a zero-knowledge proof reading the collateral price, outstanding debt, and liquidation threshold from on-chain state, asserting $\text{HF} < 1$. A liquidation leaf is committed in the epoch's liquidation root. Detection is identity-agnostic and capital-free: the prover reveals nothing beyond the breach.

Grace period (epoch boundary). Between detection and seizure, the borrower may add collateral or partially repay debt to restore $\text{HF} > 1$. The epoch duration (configurable per market) determines the length of this window.

Seizure (epoch $n+1$). A liquidator provides a Merkle proof against the liquidation root, repays outstanding debt R via the debt token contract, and seizes collateral worth $R \cdot (1 + \beta)$ from the CDP, where β is the liquidation bonus fixed at origination. For liquidity-forwarding loans, the seizure hook chain first unwinds the external venue position. Partial liquidation repays the minimum R that restores the health factor above 1; full liquidation repays the entire debt and seizes all remaining collateral. The bonus compensates the liquidator for price risk during the grace period. Collateral moves only after debt repayment is recorded on-chain.

Because each loan is isolated, bad debt cannot cascade across positions. Shortfalls reduce vault NAV through impaired debt-token value (Section 6).

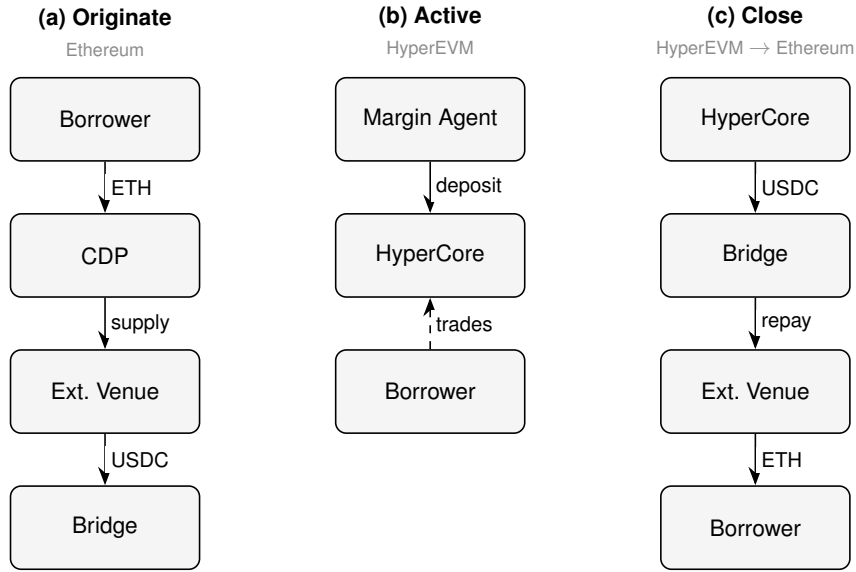


Figure 3: Prime Trade lifecycle. **(a)** Borrower pledges ETH; CDP supplies to the external venue; USDC bridged to destination. **(b)** Margin agent deposits into HyperCore; borrower trades (dashed). **(c)** Capital recalled via bridge; venue repaid; collateral released.

6 The Harvest Vault

The Harvest Vault’s operator originates loans, responds to lifecycle events, and manages portfolio concentration. All actions are bounded by a publicly auditable mandate; compliance is attested per-epoch by a zero-knowledge proof and enforced through fee-gating. The vault is the canonical lending book: when it has sufficient idle capital, it funds loans directly at the full fixed rate (Section 4); when demand exceeds capacity, it routes through external venues and commits only hedging reserves. The mandate, not operator reputation, is the trust anchor for deployed capital.

6.1 Operator and Mandate

The operator deploys vault capital by submitting solver quotes to the order book (Section 3), competing on terms like any other solver. The operator's strategy is bounded by a **mandate**: a policy committed on-chain as a hash at vault deployment. Each mandate specifies:

- **Allowed collateral set**: asset addresses the operator may accept as loan collateral.
- **Per-collateral LTV ceiling**: maximum loan-to-value ratio the operator may offer on each collateral type.
- **Per-tenor rate floor**: minimum annualized rate the operator must charge; violation forfeits the epoch fee rather than reverting the loan.
- **Portfolio concentration caps**: maximum exposure per borrower, per collateral type, and per market as a fraction of vault NAV.
- **Liquidation reserve requirement**: minimum idle capital maintained at all times for liquidation response and withdrawal settlement.

A mandate constrains what the operator *can* do, not what they *must* do; within its bounds, the operator retains full discretion on timing, counterparty selection, and position sizing.

6.2 Compliance Proof

Each epoch, the operator produces a zero-knowledge compliance proof that the vault's actions respected the mandate committed at deployment. At a high level, the proof is generated from the prior vault state, the epoch action log, the resulting vault state, and the mandate hash. It verifies that funded loans, hedging reserves, external venue choices, concentration exposure, rate floors, LTV ceilings, and liquidation reserves remained within the mandate's bounds.

The proof outputs a compliance fact registered in the facts registry and consumed by the fee-gating mechanism (Section 6.3). The detailed proof format, public inputs, and fact verification flow are part of the settlement-program framework described in Section 9. An invalid or missing compliance proof does not halt loan settlement or affect borrowers; it only prevents the operator from collecting the epoch fee.

6.3 Fee-Gating

Operators earn a spread fee escrowed during each epoch. A valid compliance fact for the corresponding vault and epoch releases the escrow; otherwise it flows back into vault NAV as LP uplift. LP-facing yield is not contingent on the compliance proof: LPs always receive their share of interest and buffer surplus.

6.4 NAV and Shares

LPs deposit USDC and receive ERC-20 shares. The vault's net asset value is

$$\text{NAV} = U_{\text{idle}} + \sum_j v_j \cdot n_j + B_{\text{escrow}} + I_{\text{accrued}} - W_{\text{pending}}, \quad (6.1)$$

where U_{idle} is undeployed USDC, $v_j \cdot n_j$ is the claim value (Equation 10.1) times quantity for each held debt token j , B_{escrow} is escrowed hedging buffers, I_{accrued} is interest that has elapsed chronologically but not yet been verified by a ZK proof (and thus not yet reflected in v_j), and W_{pending} is USDC ring-fenced for pending withdrawals. Share price is NAV per outstanding supply, with a virtual offset against inflation attacks. On vault-funded loans, the vault earns the full fixed-rate coupon; on liquidity-forwarding loans, the vault earns buffer surplus. Impairments reduce NAV directly via share-price adjustment.

6.5 Epoch Withdrawal

Instant withdrawal is incompatible with capital in active positions. The vault queues exits in four steps:

1. **Request.** The LP burns vault shares and receives *withdrawal receipt* tokens for the current epoch.
2. **Epoch elapses.** After the epoch duration, anyone can advance the epoch.
3. **Settlement.** A permissionless call snapshots the share price for all pending requests in that epoch and ring-fences the corresponding USDC, removing it from the vault's deployable idle balance.
4. **Redemption.** The LP burns receipt tokens and receives the ring-fenced USDC, net of a flat redemption fee.

Until settlement, the LP's redemption value tracks vault NAV (gains and losses included). The epoch delay prevents withdrawing at a stale NAV before a loss lands.

Table 1 decomposes risk by participant and origination path.

7 Risk Quantification

The Harvest Vault's mandate (Section 6) sets the outer boundaries (collateral classes, LTV ceilings, rate floors, concentration caps). The risk engine sets the per-loan terms within those boundaries. This section specifies how DALOC converts risk channels (Table 1) into loan-level loss distributions, underwriting terms, reserve requirements, and liquidation-assurance commitments before capital is deployed.

Each loan is treated as a collateralized credit instrument, not merely as an exposure to collateral volatility. Structural credit models show that lender risk depends on the relation between collateral value, debt, maturity, and recovery,

Party	Risk borne	Why	Compensation
Borrower	Collateral decline	Seeks leverage	Fixed-rate USDC
Ext. venue	Smart-contract, util.	Money market	Variable rate
Rate buffer	Variable > fixed	Rate transform	Surplus
Conv. reserve	Swap cost > escrow	Execution risk	Escrow $\sim 95\%$
Risk House	Liquidation shortfall	Backstop capital	Fee, gated on policy
Vault LP	Default, exhaustion	Absorbs residual	Coupon or surplus
Operator	Mandate violation	Active mgmt	Fee, gated on proof

Table 1: Risk decomposition. Each risk channel is routed to the participant with the comparative advantage to bear it. External venue, rate buffer, and conversion reserve rows apply only to liquidity-forwarding loans; vault-funded loans skip directly from borrower to Risk House and vault LP. Risk House backstop is detailed in Section 7.

not on volatility alone [22]. DeFi lending markets add endogenous funding and liquidity risk: interest rates, utilization, and market liquidity change with protocol conditions and can be correlated across venues during stress [17]. Recent work models decentralized lending contracts as option-like claims whose lender payoff depends on LTV, liquidation rules, transaction costs, and hedging feasibility [31]. Vault-based credit adds another layer: depositors bear losses through liquidation failure, oracle execution divergence, liquidity constraints, governance limits, and smart-contract risk [35]. For this reason, DALOC models the full economic loss distribution after reserves, recovery, and backstop commitments, rather than using volatility or liquidation probability as a standalone proxy for lender risk.

7.1 Risk Objective and Loss Definition

The risk engine constrains economic loss to the lender, the Harvest Vault, and any designated liquidation backstop under both ordinary and stressed conditions. A liquidation event is not itself a default. A position may be liquidated without lender loss if the debt is repaid in full, while lender loss can arise when liquidation is delayed, market depth is insufficient, execution costs are high, or funding and settlement costs exhaust loan-specific reserves. Empirical work on DeFi liquidations shows that liquidation incentives, collateral discounts, and liquidator behavior materially affect realized outcomes [25, 26]. Recent work on liquidation dynamics further shows that transaction fees, oracle-extractable value, and market microstructure can affect whether a liquidation protects lenders or amplifies loss [29]. Vault-level credit analysis therefore treats liquidation mechanics as only one component of depositor loss, alongside oracle, liquidity, governance, and infrastructure risks [35].

For each loan i , let ρ_i denote a realized scenario path. A scenario path includes collateral prices, funding rates, venue liquidity, oracle updates, execution costs,

bridge or settlement latency, and liquidation outcomes over the loan horizon. Let $\text{EAD}_i(\rho_i)$ be the exposure at close-out: contractual debt, accrued obligations, funding deficits, execution costs, and settlement costs under scenario ρ_i . Let $R_i^{\text{pre}}(\rho_i)$ be pre-backstop recovery: borrower repayments, collateral proceeds, and loan-specific reserves available before any Risk House payout. The gross economic loss before backstop support is

$$L_i^{\text{gross}} = \max(\text{EAD}_i(\rho_i) - R_i^{\text{pre}}(\rho_i), 0).$$

If loan i is covered by Risk House h with an enforceable liquidation-assurance commitment $A_{h,i}$, the House payout and residual lender loss are

$$\begin{aligned} L_i^{\text{house}} &= \min(L_i^{\text{gross}}, A_{h,i}), \\ L_i^{\text{net}} &= \max(L_i^{\text{gross}} - A_{h,i}, 0). \end{aligned}$$

Thus L_i^{gross} is the loss before Risk House support, L_i^{house} is the amount absorbed by the designated Risk House, and L_i^{net} is the residual loss borne by the lender or vault. A loan can therefore be liquidated successfully without lender loss when recovery covers the debt; a Risk House absorbs shortfall up to its commitment; residual loss, if any, falls to the vault or lender according to the market's loss waterfall.

The model separately reports three quantities. The liquidation probability is the probability that the loan breaches its health factor and enters liquidation. The loss probability is the probability that residual economic loss is positive, $\Pr[L_i^{\text{net}} > 0]$. Loss given default is the expected residual loss conditional on a positive loss event,

$$\text{LGD}_i = \mathbb{E}[L_i^{\text{net}} \mid L_i^{\text{net}} > 0].$$

At portfolio level, losses are aggregated under the same joint scenario, preserving dependence through common market, liquidity, oracle, funding, bridge, and operational shocks. Expected shortfall is used as the primary tail-risk measure because it captures the severity of losses beyond a quantile and supports portfolio aggregation and constrained optimization [3, 28]. For a portfolio loss random variable L , the expected shortfall at confidence level α is

$$\text{ES}_\alpha(L) = \mathbb{E}[L \mid L \geq \text{VaR}_\alpha(L)],$$

with the confidence level, liquidation horizon, stress set, and admissibility bounds disclosed by each Risk House and bounded by the DALOC Risk Kernel introduced in Section 7.4.

7.2 Dynamic Underwriting

Every quote is produced from the current state of the collateral, principal asset, funding path, liquidation venues, and existing portfolio. The quote specifies at least

$$q_i = (N_i, \lambda_i, \theta_i, r_i, T_i, B_i, h_i, \pi_{h_i}), \quad (7.1)$$

where N_i (gross principal), λ_i (origination LTV), and θ_i (liquidation threshold) define the position; r_i (fixed rate) and T_i (term) define the economics; B_i (reserve vector) sizes the hedging; and h_i, π_{h_i} identify the designated Risk House and its policy hash.

Principal limits, rate, liquidation threshold, reserve sizing, and admissibility are outputs of the underwriting process, not protocol constants. Fixed contractual terms are not repriced ex post; a deterioration in risk may trigger a top-up request, refinancing, or liquidation according to the market’s pre-committed rules.

7.3 Scenario Generation

Risk is estimated under the real-world probability measure because the objective is capital adequacy, not only no-arbitrage valuation. The scenario generator reproduces volatility clustering, jumps, heavy tails, and the tendency of liquidity, funding rates, and operational latency to worsen together during stress [7, 10, 17, 19, 33]. Challenger models (alternative specifications run against the primary model to detect divergence) run in parallel; parameter uncertainty is converted into conservative add-ons.

Each simulated path replays actual loan cash-flow logic (accrual, liquidation detection, epoch delay, and recovery), producing a distribution of economic loss, not merely of collateral prices. Recovery is modeled as endogenous to trade size, venue depth, and liquidator economics [25, 26, 29, 32]. For liquidity-forwarding loans, reserves are sized so that under modeled stress scenarios the protocol’s two-epoch liquidation completes before the venue’s instant trigger fires; tail scenarios beyond the modeled set may still produce a gap (Section 12).

7.4 Provable Risk Engine

The risk engine separates computation into two layers: a precomputation kernel and a serving path. The precomputation kernel is the *DALOC Risk Kernel*: a hash-committed risk surface that encodes the heavy simulation output used to bound quote generation, reserve sizing, and portfolio aggregation. The serving path answers individual quote requests against this committed kernel and produces a zero-knowledge proof of correct computation.

Precomputation kernel. Simulation results are materialized into a multi-dimensional tensor indexed by market-driven and loan-level parameters (volatility regime, LTV, rate, term). Each cell stores break-even quantities, liquidation probabilities, loss quantiles, reserve requirements, and admissibility bounds at which the protocol’s expected net loss is zero or better. Only the computationally heaviest quantities reside in the Risk Kernel; all remaining quote parameters are derived in the serving path. Recomputation happens

periodically from fresh market data; between updates, the same committed kernel serves all quotes for a given collateral/principal pair.

Serving path. When a borrower requests a quote, the serving layer maps the request into the kernel’s parameter space and searches for the optimal $(\lambda_i, \theta_i, r_i, B_i)$ that satisfies the borrower’s intent while remaining break-even or better for the lender. Interpolation over the discrete grid yields a continuous surface; optimization proceeds over it without Monte Carlo at serving time, reducing latency by orders of magnitude. Dedicated kernels serve rate-buffer and collateral-conversion-reserve sizing using the same procedure.

ZK provability. The serving computation (kernel lookup, interpolation, and optimization) is deterministic given the kernel snapshot and borrower parameters. DALOC produces a zero-knowledge proof attesting that the quote was correctly derived from the declared kernel hash and risk policy. Settlement contracts verify this proof alongside the market’s state-transition fact, ensuring that not only the executed loan terms but the underwriting logic that produced them is cryptographically auditable. Kernel precomputation, which involves stochastic Monte Carlo simulation, sits outside the proof boundary; what is proven is the deterministic path from a committed kernel to a specific quote.

7.5 Funding and Execution Reserves

For liquidity-forwarding loans, the engine sizes separate reserves for funding mismatch and execution cost. The rate buffer covers the expected shortfall of the maximum pathwise funding deficit over the loan term; the collateral conversion reserve covers stressed round-trip execution costs. Both are calibrated jointly so that a collateral crash, higher utilization, thinner liquidity, and longer settlement delay can occur in the same path [5, 6, 9, 23].

External-venue liquidity stress is treated as a first-class scenario: simulations include high utilization, borrow-rate spikes, reserve pauses, delayed withdrawals, chain downtime, bridge delay, and failed or partial unwinds. A route is rejected when the required reserve, liquidation latency, or venue exposure exceeds the market policy, Risk House policy, or Harvest Vault mandate.

7.6 Pricing

Each quote is priced from expected gross credit loss, the incremental tail capital consumed by each balance sheet in the loss waterfall, expected hedging costs, and operating costs [6, 18, 31]. Each Risk House optimizes its own objective within constraints set by the DALOC Risk Kernel and its published policy; quote competition selects the best compatible terms among feasible offers.

7.7 Portfolio Aggregation and Concentration Risk

Each new loan is assessed against the existing book, not on a standalone basis. Portfolio aggregation is performed against the current DALOC Risk Kernel (Section 7.4), which provides the committed risk surface used for quote generation, reserve sizing, and admissibility checks. Concentration limits cover counterparty, collateral asset, principal asset, external venue, bridge path, oracle source, infrastructure dependency, and market-operator dimensions.

The engine measures wrong-way risk: cases in which collateral falls while funding rates, liquidation costs, venue utilization, bridge latency, or the probability of venue failure rise. Diversification credit is recognized only when offsets remain legally, technically, and operationally available through the same close-out process [16, 34]. A portfolio that appears diversified by asset ticker may still be concentrated in the same oracle, bridge, liquidation venue, external lending protocol, or settlement operator; these shared dependencies are treated as concentration factors rather than ignored because the labels look different.

7.8 Risk Houses and Liquidation Assurance

A Risk House is an independently capitalized entity that underwrites loans and backstops liquidation shortfalls. Registration is permissionless; each House publishes a policy commitment π_h describing the risks it is willing to cover. The policy specifies admissible collateral assets, principal assets, oracle policies, external venues, hook adapters, bridge paths, liquidation routes, pricing method, LTV and tenor limits, reserve rules, concentration limits, and maximum liquidation latency. Every quote identifies the House whose capital stands behind it.

Oracle and venue admissibility are part of the House policy, not solver discretion. A Risk House may accept only oracle sources whose freshness, aggregation logic, manipulation resistance, and liquidation-price rules it can model. It may also restrict the external lending venues, swap adapters, bridge routes, and liquidation venues that can be used for covered loans. A loan using an unsupported oracle, adapter, venue, or liquidation route is not eligible for that House’s assurance commitment.

Access to shared Harvest Vault capital or a DALOC-standard designation requires compliance with the protocol-wide Risk Kernel (Section 7.4). The Risk Kernel defines minimum admissibility and risk constraints for standardized markets: collateral support, oracle quality, venue support, liquidity depth, liquidation-latency bounds, maximum LTV, minimum reserves, concentration limits, and stressed execution assumptions. A compliant Risk House may be stricter than the kernel but not weaker. This prevents a House from competing by accepting unsupported oracles, thinly traded collateral, fragile bridge paths, or external venues whose liquidity and liquidation mechanics are not modeled.

The design draws on central counterparty literature (backstop capital modeled as loss-bearing structure including wrong-way risk [2]), clearing and netting models (centralized risk absorption helps but depends on waterfall and margin design [11, 13]), and DeFi liquidation research (liquidation mechanisms and curator policies materially affect outcomes and require transparent commitments [8, 27, 34]).

Each covered loan carries an assurance commitment sized to the expected shortfall of the liquidation gap under tail and stress scenarios; capital supporting multiple loans is modeled jointly. The assurance commitment is loan-specific: it is attached to the oracle policy, collateral asset, venue path, liquidation route, and reserve assumptions used in the quote. If any of these assumptions changes, the loan must be refinanced, re-underwritten, or treated under the market's pre-committed risk rules.

The default waterfall is:

1. borrower repayments and collateral value;
2. loan-specific funding and execution reserves;
3. permissionless liquidation or competitive auction;
4. loan-specific Risk House assurance commitment;
5. Risk House general first-loss capital and vested fee escrow;
6. any narrowly defined DALOC catastrophe layer;
7. residual lender or vault impairment.

Public liquidators receive the first execution window; the Risk House acts as committed buyer only if that route fails. Normal underwriting losses consume House capital; slashing is reserved for mandate violations, false proofs, or failure to honor a committed backstop. DALOC may operate a canonical Risk House, but the core protocol does not mutualize the routine losses of every third-party House.

7.9 Operational Risk and Model Governance

Market and funding risks are estimated probabilistically where sufficient data exist. Sparse infrastructure failures (smart-contract, bridge, oracle, chain-halt, proof-system) are controlled through deterministic scenarios, exposure caps, and conservative add-ons rather than spuriously precise event frequencies. Each quote records the data snapshot, model hash, and calibration window used to produce it; recalibration is triggered when realized conditions deviate materially from model assumptions [4, 35].

8 Cross-Chain Exposure

Cross-chain operation is the default; same-chain loans are a special case. Two concerns arise at different layers.

Authorization (protocol-level): proving that collateral was pledged on the collateral chain so the principal chain’s contracts can act on it. DALOC uses ZK storage proofs via the Herodotus Data Processor (HDP) [20], combining storage proofs with MMR-backed chain history into a single verifiable claim.

Token movement (solver-level): physically moving the borrowed asset to its destination. Solvers choose the bridge path from the market’s admissible set; bridge hooks compose movement into the origination hook chain.

8.1 Authorization Flow

Consider a borrower who pledges WETH on Ethereum and requests USDC on Arbitrum. The collateral-chain issuer records the pledge; a settlement program reads the CDP’s storage slot from Ethereum’s state root via HDP (a Merkle inclusion proof against the state trie, attested by an MMR over canonical block headers), producing a verifiable claim that the collateral is locked. On Arbitrum, the debt issuer validates this claim through the facts registry and issues the debt token.

Each chain’s issuer advances its epoch independently; coordination relies on shared market identifiers and deterministic addressing, not real-time cross-chain messaging. Bridge latency affects capital delivery but not settlement correctness; a delayed bridge stalls execution, never produces an invalid state.

8.2 Deterministic Addressing

Identical hub addresses on every supported chain, combined with deterministic CREATE2 loan contracts, allow collateral on chain A and debt on chain B to reference the same loan identifier. Adding a new destination chain requires only a bridge adapter whitelisted on the hub.

8.3 Hyperliquid Margin Agent

For Prime Trade, the margin pipeline (Section 5) routes borrowed USDC to a per-loan *margin agent* on HyperEVM [21] at a CREATE2 address computed at quote time. Capital enters HyperCore as margin, never the borrower’s wallet. Permitted actions are restricted to perpetual trading; withdrawals, spot transfers, and external lending are blocked.

On close or liquidation, the margin agent recalls capital from HyperCore and bridges it back to Ethereum. Collateral never crosses to HyperEVM; only USDC round-trips. Figure 3 illustrates the full origination, active trading, and close cycle.

8.4 Effective Leverage

For Prime Trade, the borrowing LTV combined with venue leverage creates a multiplicative effect. For collateral worth C deposited at LTV λ with venue

leverage ℓ , the notional controlled is

$$N = C \cdot \lambda \cdot \ell, \quad (8.1)$$

and the effective leverage relative to the collateral is $\lambda_{\text{eff}} = N/C = \lambda \cdot \ell$.

At $\lambda = 62.5\%$ LTV with $\ell = 5\times$ venue leverage, $\lambda_{\text{eff}} = \lambda \cdot \ell \approx 3.1\times$; this yields substantial notional controlled without selling the underlying. Collateral remains on Ethereum throughout.

9 Settlement and Verification

9.1 Epoch Mechanics

Time advances in *epochs*: bounded settlement windows in which the off-chain engine matches intents, runs state-transition programs, and commits a snapshot on-chain. Epoch duration is configurable per market, trading latency against proof cost.

Each snapshot carries a set of Merkle roots encoding transfers, market state, debt records, liquidation events, and contract deployments.¹ Posting a new epoch requires a valid state-transition fact in the facts registry, attesting correct transition from the previous snapshot to the new one.

Examples of facts that can support an epoch transition include: a matching fact showing that accepted borrower intents and solver quotes were criteria-compatible and that no better compatible quote was skipped; an origination fact committing the transfer and hook leaves for a new loan; a liquidation-detection fact asserting that a loan's health factor is below one under the market's oracle program; a close or refinancing fact verifying repayment, current interest accrual, and reserve accounting; and a vault-compliance fact attesting that operator actions respected the Harvest Vault mandate. Only the state-transition fact advances the market epoch; the other facts justify specific leaves or accounting updates included in that transition.

Execution is inclusion-proof driven: anyone can execute a committed leaf by supplying a Merkle proof against the epoch root stored on-chain.

Each epoch advance processes three concurrent generations: finalizing loans from two epochs prior, settling principal from one epoch prior, and matching new intents. When the solver is the vault operator (the common case), all three compress into a single epoch via an atomic multicall; the multi-generation pipeline applies when independent solvers require cross-chain coordination between steps.

¹Internal state uses ZK-friendly Merkle structures; on-chain roots use keccak-hashed binary trees.

9.2 Settlement Programs

On-chain contracts are settlement executors: they verify Merkle proofs and execute committed transfers and hook chains but contain no market-specific logic. Market behavior (interest computation, liquidation detection, oracle selection, collateral policy, and mandate compliance) is computed by zero-knowledge programs off-chain. Those programs output facts: hash-bound statements tying a program hash, market identifier, chain, epoch, prior state, new state, and any computed values to a verified proof.

The contracts do not recompute the market logic. Instead, they reconstruct the expected fact hash from their fixed parameters and the submitted leaf or state transition, check that the facts registry marks that fact as valid, and apply the corresponding transfer, accrual, liquidation, or epoch update only if the fact is present. What is verified on-chain is therefore not an oracle feed or risk model directly, but the existence of a valid proof that the declared program produced the declared output from the declared inputs.

Programs split by economic cadence. **Matching** is one-time per loan: transfers, debt issuance, CDP authorization, and criteria-compatibility evaluation (Section 3). **Interest accrual** is periodic per debt per time window. **Liquidation** is event-triggered when health breaches produce liquidation leaves. **Close** is terminal after repayments and accrual catch up. **Compliance** is periodic per vault epoch, attesting operator behavior against the mandate (Section 6).

All programs submit results through a single *facts registry* that accepts a fact hash and returns whether it is valid. Two on-chain fact structures carry all market state. A **state-transition fact** authorizes an epoch advance, binding the settlement program, chain, issuer, and the previous and next epoch roots (each encoding the full set of Merkle roots). An **interest fact** authorizes accrual on one loan's debt, binding the rate program, chain, debt identifier, accrual window, and computed interest amount. The on-chain contract reconstructs the expected fact from its fixed parameters, checks the registry, and applies values only if valid. Matching, origination, liquidation-detection, close, and compliance proofs produce state-transition facts that incorporate their respective logic; the two on-chain structures are the common execution interface through which all program types take effect.

9.3 Market and Loan Contracts

Each market has two contracts. The **collateral manager issuer** (collateral chain) advances epoch state, executes Merkle-authorized transfers, and deploys per-loan collateral managers. The **debt issuer** (principal chain) advances epoch state and deploys per-loan issued debt tokens. Neither contains market-specific logic.

Each loan also has two contracts. The **collateral manager** (CDP) holds pledged assets and executes hook-composed pipelines; permitted operations are defined

by the market's ZK programs. The **issued debt** is a per-loan ERC-20 tracking accrual via verified facts and partitioning repayment into principal reserve, interest reserve, and protocol fees (Section 10).

Chain downtime is treated as a liveness failure, not a correctness failure. The affected issuer or adapter cannot advance epochs, execute hooks, or process lifecycle events until block production resumes; the market resumes from its last valid state root. During the halt, economic risk can continue to move on other chains (oracle prices change, margin positions gain or lose value), so markets treat chain downtime as a modeled liveness risk in their admissibility checks, liquidation-latency bounds, and Risk House commitments.

9.4 Oracle Freedom and Permissionless Markets

Oracle logic is embedded in the market's zero-knowledge programs. Programs read price or rate data from on-chain contracts via storage proofs, enforcing freshness inside the proof rather than trusting off-chain feeds. Markets can be created permissionlessly: the protocol verifies program outputs but does not audit program logic. Assessing a market's risk properties is the participant's responsibility.

9.5 System Invariants

Three invariants hold across all states:

1. **Token conservation.** For every asset, total balances across all market contracts equal deposited collateral plus supplied principal minus withdrawals. No operation creates or destroys tokens.
2. **Epoch monotonicity.** Each market's epoch advances strictly by one, only when the facts registry contains a valid state-transition fact linking the previous and next roots. No epoch can be skipped, replayed, or forked.
3. **Position isolation.** Each loan's contracts are independently deployed, independently liquidatable, and share no mutable state with any other loan. One position's insolvency cannot reach into another's contract state. (Economic correlation through a shared creditor, such as the Harvest Vault, may affect NAV but does not violate contract-level isolation.)

10 Position Tokens

Each funded loan produces a tokenized position that represents the lender's claim, carries accrual, and enforces solvency rules as a transferable ERC-20.

10.1 Debt Tokens

Each funded loan issues a transferable ERC-20 whose face value is one unit of the principal asset per token. The token is the lender's economic position. The

claim value per token v (the amount redeemable at maturity) is

$$v = 1 + I_{\text{cum}}, \quad (10.1)$$

where I_{cum} is the cumulative interest accrued per token, rising as interest is verified. Interest accrues linearly (simple interest, not compound), so the borrower's total obligation at maturity for principal P over term T is

$$D = P \left(1 + r \frac{T}{T_{\text{year}}} \right), \quad (10.2)$$

and the per-period increment used by the on-chain accrual program is

$$\Delta I = P r \frac{\Delta t}{T_{\text{year}}}, \quad (10.3)$$

where P is the outstanding principal in token units, r the annualized rate, Δt the accrual period in seconds, and $T_{\text{year}} = 365.25 \times 86,400$. Linear accrual is a deliberate choice: the borrower's obligation is fully determined at origination, with no path-dependent compounding. Each accrual requires a valid interest fact verified through the facts registry (Section 9).

Repayment proceeds enter a principal reserve. A subsequent accrual step, backed by a proof tied to the market's rate program, splits the interest portion into a protocol fee and an interest reserve. Holders redeem by burning tokens and receiving principal plus their share of the interest index.

A strict accounting identity holds at all times, ensuring no principal, interest, or fee is created or destroyed:

$$R_{\text{principal}} + R_{\text{interest}} + F_{\text{protocol}} = B_{\text{token}}, \quad (10.4)$$

where $R_{\text{principal}}$ is the redeemable principal reserve, R_{interest} the interest reserve, F_{protocol} the accrued protocol fees, and B_{token} the contract's token balance. Redemptions are blocked until interest for the latest period is accrued, preventing exits at a stale index. A loan can be marked closed only when accrual is current and the principal reserve covers all outstanding tokens.

Debt tokens from criteria-restricted markets carry transfer-time criteria (e.g. "KYC tier ≥ 2 ") enforced at the hub-routed transfer step, ensuring restrictions persist after origination. When the Harvest Vault is creditor, vault shares (Section 6) reflect the aggregate claim value across all held debt tokens. Debt tokens are ERC-20 with transfer routing through the hub; this hub routing enables criteria enforcement while preserving standard ERC-20 composability for unrestricted tokens.

11 Governance and Trust Assumptions

The protocol's trust surface is deliberately narrow. Market parameters are fixed at creation and identified by their hash; every epoch advance requires a valid

zero-knowledge proof registered in the facts registry (Section 9). Trust concentrates in the hub contract, which governs all capital flows, and in the liveness of off-chain infrastructure.

Beyond these trust surfaces, the protocol minimizes dependence on any single party. Markets are independent state machines (Section 3) that advance their own epochs without shared global state; deployment is permissionless, and so is participation as a solver, liquidator, or epoch executor. A forced-inclusion path through the settlement layer ensures that no operator can permanently censor intents or block exits.

Safety is structural: the system invariants of Section 9 (token conservation, epoch monotonicity, position isolation) hold across all reachable states, and no state transition can occur without a valid zero-knowledge proof. Mandate compliance is enforced through fee-gating rather than execution-blocking (Section 6). Liveness depends on each market's operator and on the chains, bridges, provers, and external venues used by that market; a configurable timeout bounds how long a market can stall before a fallback operator may step in.

The architecture targets custody recoverability without reliance on the original off-chain operator. Users should be able to submit cancellations, forced-inclusion requests, withdrawals, or recovery actions directly to the relevant on-chain contracts; for markets deployed on L2s, this may rely on that L2's L1 inbox, escape hatch, or forced-transaction mechanism. This does not mean that every economic action is immediately available from L1: liquidation, oracle updates, bridge movement, and external-venue unwinds still depend on the relevant chains and infrastructure being live. Each market defines its own data-availability and recovery policy, and the architecture admits progressive trust reduction as markets mature.

11.1 Protocol Administration

A multisig with timelock governs protocol contracts, which are upgradeable. Upgradeability permits response to critical vulnerabilities at the cost of trust in the multisig. Because core contracts control capital flows and settlement execution, an upgrade can alter routing and pause behavior across every market simultaneously. Pause and bridge-adapter whitelisting are also multisig actions; cross-chain routes are permissioned even though markets are not.

11.2 Liveness Dependencies

Settlement correctness is enforced by zero-knowledge proofs (Section 9), but liveness is not: if a market's off-chain infrastructure halts, no new epochs advance for that market. Existing positions remain safe (collateral stays locked, debt is unchanged, intent cancellation remains available), but undercollateralized positions go undetected until epochs resume.

Each market defines a liveness timeout. If no epoch advances within that window, users may withdraw deposited capital, and the market may permit a fallback operator to resume epoch production.

LTV limits, rates, and buffer sizes are produced off-chain by the risk engine. The serving computation is cryptographically verified via zero-knowledge proof (Section 7), ensuring quotes derive correctly from the declared kernel and risk policy. The mandate (Section 6) imposes outer bounds (rate floors, LTV ceilings, concentration caps) enforced through fee-gating. Neither mechanism validates the underlying model calibration or Monte Carlo assumptions between those bounds.

Cross-chain movement depends on the selected bridge infrastructure (Section 8); the protocol inherits the bridge's liveness and censorship properties. Authorization correctness is cryptographic (ZK storage proofs over actual chain state), but proof-generation availability depends on the data provider, and bridge execution depends on the source and destination chains being live.

12 Discussion

12.1 Risks and Limitations

Smart-contract risk. The hub, issuers, CDPs, and debt tokens share common contract logic. A critical vulnerability in a widely-used contract could affect multiple markets. Formal verification covers token conservation and epoch monotonicity; broader functional coverage is an ongoing effort.

Oracle staleness. Liquidation detection depends on timely oracle updates. If oracle prices lag during a flash crash, detection is delayed and recovery proceeds may fall below debt. The two-epoch structure bounds this lag to one additional epoch window; markets with more volatile collateral should configure shorter epochs.

Proof-system failures. If the ZK prover becomes unavailable, no new epochs can advance. Existing positions are safe (funds remain in CDPs) but cannot settle new interest or originate. Recovery requires a new prover deployment. No fallback to optimistic settlement is currently implemented.

Dual-liquidation race. A liquidity-forwarding loan exposes the vault to the gap between DALOC's two-epoch liquidation and the venue's instant liquidation trigger. Reserve sizing (Section 7) attempts to make this gap negligible, but a sufficiently sharp collateral crash can trigger the venue's liquidation before DALOC's grace period completes, leaving a residual shortfall absorbed by the vault.

Cross-chain liveness. Bridge delays or outages stall capital delivery on Prime Trade origination and close. Positions remain valid but economically frozen

until the bridge resumes. Markets that rely on a single bridge path inherit its liveness profile.

Host-chain downtime. DALOC markets inherit the liveness of the chains on which their contracts are deployed. A chain halt delays epoch advances, hook execution, and liquidations while economic risk continues to move elsewhere. Markets on L2s additionally inherit the rollup’s recovery properties. Chain downtime is a modeled liveness risk (Section 9), not a correctness failure.

External-venue liquidity and liveness. Liquidity-forwarding loans depend on external venues for borrowing, repayment, and liquidation timing. If a venue pauses, has insufficient liquidity, or liquidates faster than DALOC’s two-epoch process, the loan may enter a pending-unwind state and residual losses flow through the loss waterfall (Section 7). Reserve sizing makes this risk explicit and priced but cannot remove it.

Prime Trade venue downtime. Prime Trade positions depend on HyperEVM and HyperCore remaining live. If the destination venue stalls, the margin agent cannot rebalance, stop out, or recall capital on schedule. The position remains constrained by its program, but economic exposure changes while execution is unavailable. Venue downtime is modeled in the Risk Kernel and Risk House policy.

Stablecoin and bridge centralization. For USDC-based markets, the principal asset issuer (Circle) can freeze balances at specific addresses. If a CDP, vault, or margin agent address is blacklisted, the affected positions cannot settle normally. This concentration risk is mitigated by market diversity (alternative principal assets and bridge adapters may serve separate markets) but cannot be eliminated while a single issuer’s token is the primary path.

12.2 Future Capabilities

The architecture is designed for staged extension. Near-term additions include:

- **Risk House ecosystem.** The architecture already admits permissionless Risk House registration (Section 7); at scale, multiple independent houses compete on terms and backstop specific collateral classes. The kernel bounds prevent a weak house from degrading system-wide safety.
- **Under-collateralized credit.** On-chain reputation, verifiable off-chain creditworthiness (e.g. institutional attestations), and cross-protocol track records enable LTV thresholds above 100% for qualified borrowers under tighter Risk House oversight.
- **Cross-margining.** Correlated positions under the same borrower and same Risk House can share margin via a portfolio-level health factor, improving capital efficiency without weakening per-position isolation at the settlement layer.

- **Options and nonlinear collateral.** Collateral-class adapters for options and other nonlinear claims, valued at stressed executable close-out value rather than oracle mark, extend the protocol beyond spot and yield-bearing tokens.
- **Multi-venue Prime Trade.** Margin agents generalize beyond HyperEVM to additional perpetual venues, each as a separate bridge adapter and constraint set.
- **Operator ecosystem.** The architecture supports multiple independent vault operators, each running a vault instance with a distinct mandate (one conservative with high-grade collateral and short tenors, another aggressive with broader collateral and higher concentration limits). Competition between operators for LP capital creates pressure toward better risk-adjusted returns. Per-market settlement operators may independently rotate or compete, mirroring vault-level competition at the settlement layer.
- **Institutional access.** Criteria-based matching (Section 3) already enables regulated counterparties to participate without a permissioned fork. Realizing this in production requires only attestation issuers and criteria definitions, not protocol modification.
- **Maturity management.** Refinancing (Section 5) already allows mid-term rollovers. At scale, automated maturity laddering (where the operator pre-matches refinancing quotes ahead of expiry) can smooth redemption waves, reduce cliff-event risk, and improve capital utilization across the loan book.

12.3 Conclusion

DALOC unifies collateralized credit, cross-chain capital coordination, and risk quantification within a single settlement layer. The Harvest Vault funds loans directly under a proof-attested mandate; external venues provide overflow capacity beyond the vault’s balance sheet. Fixed-rate, fixed-term loans; leveraged perpetual trading without selling the underlying; and passive yield from a portfolio of fixed-rate loans share a common matching and settlement infrastructure.

Criteria-based matching lets restricted and unrestricted counterparties share a single liquidity surface. Zero-knowledge proofs enforce settlement correctness and underwriting integrity; mandate compliance is enforced through fee-gated compliance proof. Per-position and per-market isolation bound the blast radius of defaults. The protocol presents an architecture for the transition from pool-based, variable-rate, single-chain lending to structured, provable, multi-chain credit.

References

- [1] Aave. Aave V4 overview. Technical report, Aave, 2026. URL <https://github.com/aave/aave-v4/blob/main/docs/overview.md>.
- [2] Matthias Arnsdorf. Central counterparty risk, 2012. URL <https://arxiv.org/abs/1205.1533>.
- [3] Philippe Artzner, Freddy Delbaen, Jean-Marc Eber, and David Heath. Coherent measures of risk. *Mathematical Finance*, 9(3):203–228, 1999. doi: 10.1111/1467-9965.00068.
- [4] Massimo Bartoletti and Enrico Lipparini. A theory of lending protocols in DeFi, 2025. URL <https://arxiv.org/abs/2506.15295>.
- [5] Mahsa Bastankhah, Viraj Nadkarni, Xuechao Wang, and Pramod Viswanath. AgileRate: Bringing adaptivity and robustness to DeFi lending markets, 2024. URL <https://arxiv.org/abs/2410.13105>.
- [6] Bastien Baude, Damien Challet, and Ioane Muni Toke. Optimal risk-aware interest rates for decentralized lending protocols, 2025. URL <https://arxiv.org/abs/2502.19862>.
- [7] Carlos Castro-Iragorri, Julian Ramirez, and Sebastian Velez. Financial intermediation and risk in decentralized lending protocols, 2021. URL <https://arxiv.org/abs/2107.14678>.
- [8] Tarun Chitra. A curatory tale: Logarithmic regret in DeFi lending via dynamic pricing, 2025. URL <https://arxiv.org/abs/2503.18237>.
- [9] Tarun Chitra, Peteris Erins, and Kshitij Kulkarni. Attacks on dynamic DeFi interest rate curves, 2023. URL <https://arxiv.org/abs/2307.13139>.
- [10] Jonathan Chiu, Emre Ozdenoren, Kathy Yuan, and Shengxing Zhang. On the fragility of DeFi lending. Staff Working Paper 2023-14, Bank of Canada, 2023. URL <https://www.bankofcanada.ca/2023/02/staff-working-paper-2023-14/>.
- [11] Rama Cont and Thomas Kokholm. Central clearing of OTC derivatives: Bilateral versus multilateral netting. *Statistics and Risk Modeling*, 31(1): 3–22, 2014. doi: 10.1515/strm-2013-1161. URL <https://arxiv.org/abs/1304.5065>.
- [12] DefiLlama. DefiLlama — DeFi dashboard. <https://defillama.com>, 2026.
- [13] Darrell Duffie and Haoxiang Zhu. Does a central clearing counterparty reduce counterparty risk? *The Review of Asset Pricing Studies*, 1(1):74–95, 2011. doi: 10.1093/rapstu/rar001.

- [14] Euler Labs. Euler v2 whitepaper. Technical report, Euler Labs, 2024. URL <https://docs.euler.finance/euler-v2/about/whitepaper>.
- [15] Mathis Gontier Delaunay, Paul Frambot, Quentin Garchery, and Matthieu Lesbre. Morpho blue whitepaper. Technical report, Morpho Labs, 2023. URL <https://github.com/morpho-org/morpho-blue/blob/main/morpho-blue-whitepaper.pdf>.
- [16] Michael B. Gordy. A risk-factor model foundation for ratings-based bank capital rules. *Journal of Financial Intermediation*, 12(3):199–232, 2003. doi: 10.1016/S1042-9573(03)00040-8.
- [17] Lewis Gudgeon, Sam M. Werner, Daniel Perez, and William J. Knottenbelt. DeFi protocols for loanable funds: Interest rates, liquidity and market efficiency, 2020. URL <https://arxiv.org/abs/2006.13922>.
- [18] Joseph Y. Halpern, Rafael Pass, and Aditya Saraf. Fair interest rates are impossible for lending pools: Results from options pricing, 2024. URL <https://arxiv.org/abs/2410.11053>.
- [19] Lioba Heimbach, Eric G. Schertenleib, and Roger Wattenhofer. Short squeeze in DeFi lending market: Decentralization in jeopardy?, 2023. URL <https://arxiv.org/abs/2302.04068>.
- [20] Herodotus. Herodotus data processor: Provable on-chain data access. <https://docs.herodotus.cloud>, 2024.
- [21] Hyperliquid Labs. Hyperliquid: A performant L1 for a decentralized financial system. <https://hyperliquid.xyz>, 2024.
- [22] Robert C. Merton. On the pricing of corporate debt: The risk structure of interest rates. *The Journal of Finance*, 29(2):449–470, 1974. doi: 10.1111/j.1540-6261.1974.tb03058.x.
- [23] Viraj Nadkarni and Pramod Viswanath. Split the yield, share the risk: Pricing, hedging and fixed rates in DeFi, 2025. URL <https://arxiv.org/abs/2505.22784>.
- [24] Bhargav Nagaraja Bhatt, Paul Frambot, Quentin Garchery, Mathis Gontier Delaunay, Adrien Husson, Paul-Adrien Nicole, Adrien Laversanne-Finot, and Matthieu Lesbre. Morpho midnight whitepaper. Technical report, Morpho Labs, 2025. URL <https://github.com/morpho-org/morpho-midnight/blob/main/morpho-midnight-whitepaper.pdf>.
- [25] Daniel Perez, Sam M. Werner, Jiahua Xu, and Benjamin Livshits. Liquidations: DeFi on a knife-edge. In *Financial Cryptography and Data Security*, pages 457–476. Springer, 2021. doi: 10.1007/978-3-662-64331-0_24. URL <https://arxiv.org/abs/2009.13235>.

- [26] Kaihua Qin, Liyi Zhou, Pablo Gamito, Philipp Jovanovic, and Arthur Gervais. An empirical study of DeFi liquidations: Incentives, risks, and instabilities. In *Proceedings of the 21st ACM Internet Measurement Conference*, pages 336–350. ACM, 2021. doi: 10.1145/3487552.3487811. URL <https://arxiv.org/abs/2106.06389>.
- [27] Kaihua Qin, Jens Ernstberger, Liyi Zhou, Philipp Jovanovic, and Arthur Gervais. Mitigating decentralized finance liquidations with reversible call options, 2023. URL <https://arxiv.org/abs/2303.15162>.
- [28] R. Tyrrell Rockafellar and Stanislav Uryasev. Optimization of conditional value-at-risk. *The Journal of Risk*, 2(3):21–41, 2000. doi: 10.21314/JOR.2000.038.
- [29] Agathe Sadeghi and Zachary Feinstein. Liquidation dynamics in DeFi and the role of transaction fees, 2026. URL <https://arxiv.org/abs/2602.12104>.
- [30] Sky (formerly MakerDAO). SparkLend. Technical report, Sky, 2023. URL <https://docs.spark.fi>.
- [31] Lukasz Szpruch, Marc Sabaté Vidales, Tanut Treetanthiploet, and Yufei Zhang. Pricing and hedging of decentralised lending contracts, 2024. URL <https://arxiv.org/abs/2409.04233>.
- [32] Phoebe Tian and Yu Zhu. Liquidation mechanisms and price impacts in DeFi. Staff Working Paper 2025-12, Bank of Canada, 2025. URL <https://www.bankofcanada.ca/2025/03/staff-working-paper-2025-12/>.
- [33] Natkamon Tovanich, Myriam Kassoul, Simon Weidenholzer, and Julien Prat. Contagion in decentralized lending protocols: A case study of compound. In *Proceedings of the 2023 Workshop on Decentralized Finance and Security*. ACM, 2023. doi: 10.1145/3605768.3623544. URL <https://hal.science/hal-04221228>.
- [34] Anastasiia Zbandut and Carolina Goldstein. Institutionalizing risk curation in decentralized credit, 2025. URL <https://arxiv.org/abs/2512.11976>.
- [35] Anastasiia Zbandut and Carolina Goldstein. Vault as a credit instrument, 2026. URL <https://arxiv.org/abs/2604.17579>.